

EduTrack API — Documentación completa para desarrollo de Frontend

Stack del backend

- Java 17 · Spring Boot 3.3.5 · Spring Cloud 2023.0.3
- PostgreSQL (auth + course) · MySQL (enrollment)
- JWT Authentication · AWS S3 (imágenes)

URL base

Todos los requests van al API Gateway:

http://localhost:8080

El frontend NUNCA habla directamente con los microservicios, solo con el gateway.

Autenticación

Flujo JWT

1. El usuario hace login → recibe un token JWT
2. Cada request protegido debe incluir el header:

Authorization: Bearer <token>

Estructura del token JWT (payload)

```
{
  "sub": "email@ejemplo.com",
  "role": "ROLE_USER",
  "id": 1,
  "iat": 1234567890,
  "exp": 1234567890
}
```

Roles disponibles

- USER — usuario normal, puede inscribirse y ver cursos
 - ADMIN — puede crear/editar/eliminar cursos, categorías y lecciones
-

Rutas públicas (sin token)

- POST /api/auth/register
 - POST /api/auth/login
 - GET /api/courses
 - GET /api/courses/{id}
 - GET /api/categories
-

AUTH SERVICE — /api/auth

POST /api/auth/register

Registra un nuevo usuario. Devuelve token JWT.

Request body:

```
{
  "name": "David Farfán",
  "email": "david@example.com",
  "password": "123456"
}
```

Response 200:

```
{
```

```
"token": "eyJhbGciOi...",
"type": "Bearer",
"id": 1,
"name": "David Farfán",
"email": "david@example.com",
"role": "USER"
```

```
}
```

Errores:

- 409 — Email ya registrado
- 400 — Datos inválidos (nombre vacío, email inválido, contraseña menor a 6 chars)

POST /api/auth/login

Logín de usuario. Devuelve token JWT.

Request body:

```
{
  "email": "david@example.com",
  "password": "123456"
```

```
}
```

Response 200:

```
{
  "token": "eyJhbGciOi...",
  "type": "Bearer",
  "id": 1,
  "name": "David Farfán",
  "email": "david@example.com",
  "role": "USER"
```

```
}
```

Errores:

- 401 — Credenciales incorrectas

GET /api/auth/me

Devuelve el perfil del usuario autenticado.

Headers requeridos: Authorization: Bearer <token>

Response 200:

```
{
  "id": 1,
  "name": "David Farfán",
  "email": "david@example.com",
  "role": "USER",
  "createdAt": "2026-05-09T12:55:21.269967"
```

```
}
```

GET /api/auth/users

Lista todos los usuarios. Solo ADMIN.

Headers requeridos: Authorization: Bearer <token ADMIN>

Response 200:

```
[
  {
    "id": 1,
    "name": "David Farfán",
    "email": "david@example.com",
    "role": "USER",
    "createdAt": "2026-05-09T12:55:21.269967"
  }
]
```

COURSE SERVICE — /api/courses, /api/lessons, /api/categories**GET /api/courses**

Lista cursos paginados. Filtro opcional por categoría.

Query params:

Param	Tipo	Default	Descripción
page	int	0	Número de página
size	int	10	Elementos por página
categoryId	Long	null	Filtrar por categoría
sort	string	createdAt	Campo de ordenamiento

Response 200:

```
{
  "content": [
    {
      "id": 1,
      "title": "Java desde cero",
      "description": "Aprende Java desde los fundamentos",
      "imageUrl": "https://edutrack-covers.s3.us-east-2.amazonaws.com/covers/uuid-
imagen.jpg",
      "category": {
        "id": 1,
        "name": "Programación"
      },
      "createdAt": "2026-05-09T19:04:52.240324"
    }
  ],
  "totalElements": 10,
  "totalPages": 2,
  "size": 6,
  "number": 0,
  "first": true,
  "last": false
}
```

GET /api/courses/{id}

Detalle completo de un curso con sus lecciones.

Response 200:

```
{
  "id": 1,
  "title": "Java desde cero",
  "description": "Aprende Java desde los fundamentos",
  "imageUrl": "https://edutrack-covers.s3.us-east-2.amazonaws.com/covers/uuid.jpg",
  "category": {
    "id": 1,
    "name": "Programación"
  },
  "lessons": [
    {
      "id": 1,
      "title": "Variables y tipos de datos",
      "content": "En esta lección aprenderás...",
      "orderIndex": 1
    }
  ],
  "createdAt": "2026-05-09T19:04:52.240324"
}
```

Errores:

- 404 — Curso no encontrado

POST /api/courses

Crea un nuevo curso. Solo ADMIN.

Headers requeridos: Authorization: Bearer <token ADMIN>

Request body:

```
{
  "title": "Python para todos",
  "description": "Curso completo de Python",
  "categoryId": 1
}
```

Response 200: mismo formato que GET /api/courses/{id}

PUT /api/courses/{id}

Actualiza un curso. Solo ADMIN.

Headers requeridos: Authorization: Bearer <token ADMIN>

Request body: mismo que POST

Response 200: mismo formato que GET /api/courses/{id}

DELETE /api/courses/{id}

Elimina un curso. Solo ADMIN.

Headers requeridos: Authorization: Bearer <token ADMIN>

Response 204: No content

POST /api/courses/{id}/image

Sube imagen de portada a AWS S3. Solo ADMIN.

Headers requeridos: Authorization: Bearer <token ADMIN>

Request: multipart/form-data

Field	Tipo	Descripción
-------	------	-------------

file	File	Imagen (jpg, png, webp)
------	------	-------------------------

Response 200:

```
{
  "id": 1,
  "title": "Java desde cero",
  "imageUrl": "https://edutrack-covers.s3.us-east-2.amazonaws.com/covers/uuid-
imagen.jpg",
  ...
}
```

POST /api/courses/{id}/lessons

Agrega una lección a un curso. Solo ADMIN.

Headers requeridos: Authorization: Bearer <token ADMIN>

Request body:

```
{
  "title": "Variables y tipos de datos",
  "content": "Contenido de la lección...",
  "orderIndex": 1
}
```

Response 200:

```
{
  "id": 1,
  "title": "Variables y tipos de datos",
  "content": "Contenido de la lección...",
  "orderIndex": 1
}
```

GET /api/lessons/{id}

Detalle de una lección. Requiere autenticación.

Headers requeridos: Authorization: Bearer <token>

Response 200:

```
{
  "id": 1,
  "title": "Variables y tipos de datos",
  "content": "Contenido completo de la lección...",
  "orderIndex": 1
}
```

GET /api/categories

Lista todas las categorías paginadas.

Query params: page, size

Response 200:

```
{
  "content": [
    { "id": 1, "name": "Programación" },
    { "id": 2, "name": "Diseño" }
  ],
  "totalElements": 2,
  "totalPages": 1
}
```

POST /api/categories

Crea una categoría. Solo ADMIN.

Headers requeridos: Authorization: Bearer <token ADMIN>

Query param: ?name=Programación

Response 200:

```
{
  "id": 1,
  "name": "Programación"
}
```

ENROLLMENT SERVICE — /api/enrollments

POST /api/enrollments

Inscribe al usuario autenticado en un curso.

Headers requeridos: Authorization: Bearer <token USER>

Request body:

```
{
  "courseId": 1
}
```

Response 200:

```
{
  "id": 1,
  "userId": 1,
  "courseId": 1,
  "enrolledAt": "2026-05-09T19:38:45.912268"
}
```

Errores:

- 400 — Ya estás inscrito en este curso

GET /api/enrollments/my

Lista los cursos en los que está inscrito el usuario autenticado.

Headers requeridos: Authorization: Bearer <token USER>

Response 200:

```
[
  {
    "id": 1,
    "userId": 1,
    "courseId": 1,
    "enrolledAt": "2026-05-09T19:38:45.912268"
  }
]
```

POST /api/enrollments/progress

Marca una lección como completada.

Headers requeridos: Authorization: Bearer <token USER>

Request body:

```
{
  "courseId": 1,
  "lessonId": 1
}
```

Response 200:

```
{
  "lessonId": 1,
  "completed": true,
  "completedAt": "2026-05-09T19:39:41.431132"
}
```

Errores:

- 400 — No estás inscrito en este curso
-

GET /api/enrollments/progress/{courseId}

Devuelve el progreso del usuario en un curso específico.

Headers requeridos: Authorization: Bearer <token USER>

Response 200:

```
{
  "courseId": 1,
  "totalLessons": 3,
  "completedLessons": 2,
  "percentage": 66.7,
  "lessons": [
    {
      "lessonId": 1,
      "completed": true,
      "completedAt": "2026-05-09T19:39:41.431132"
    },
    {
      "lessonId": 2,
      "completed": true,

```

```
    "completedAt": "2026-05-09T19:45:00.000000"
  },
  {
    "lessonId": 3,
    "completed": false,
    "completedAt": null
  }
]
}
```

Errores:

- 400 — No estás inscrito en este curso

GET /api/enrollments/course/{courseId}

Lista todos los estudiantes inscritos en un curso. Solo ADMIN.

Headers requeridos: Authorization: Bearer <token ADMIN>

Response 200:

```
[
  {
    "id": 1,
    "userId": 1,
    "courseId": 1,
    "enrolledAt": "2026-05-09T19:38:45.912268"
  }
]
```

Manejo de errores

Todos los endpoints devuelven errores en este formato:

```
{
  "timestamp": "2026-05-09T19:01:15.264667600",
  "status": 403,
  "message": "Acceso denegado — se requiere ADMIN",
  "details": {}
}
```

Códigos de error comunes

Status Significado

400	Datos inválidos o regla de negocio violada
401	Sin token o token inválido/expirado
403	Sin permisos (se requiere ADMIN)
404	Recurso no encontrado
409	Conflicto (email duplicado)
500	Error interno del servidor

Flujos principales del frontend

Flujo de autenticación

1. POST /api/auth/register → guardar token en localStorage
2. POST /api/auth/login → guardar token en localStorage
3. GET /api/auth/me → obtener datos del usuario al cargar la app
4. Logout → eliminar token de localStorage

Flujo de catálogo

1. GET /api/categories → mostrar filtros
2. GET /api/courses?categoryId=1 → listar cursos filtrados
3. GET /api/courses/{id} → ver detalle con lecciones

Flujo de inscripción y progreso

1. POST /api/enrollments → inscribirse
2. GET /api/enrollments/progress/{courseId} → ver progreso actual
3. POST /api/enrollments/progress → marcar lección completada
4. GET /api/enrollments/my → listar mis cursos

Flujo de admin

1. POST /api/categories → crear categoría
2. POST /api/courses → crear curso
3. POST /api/courses/{id}/image → subir portada a S3
4. POST /api/courses/{id}/lessons → agregar lecciones
5. PUT /api/courses/{id} → editar curso
6. DELETE /api/courses/{id} → eliminar curso

Notas importantes para el frontend

1. **Token storage** — guardar en localStorage con key token
2. **Header de autorización** — siempre Bearer <token> (con espacio)
3. **Subida de imágenes** — usar multipart/form-data, NO application/json
4. **Paginación** — los listados devuelven objeto con content[], totalPages, totalElements
5. **Roles** — el campo role en la respuesta de login es "USER" o "ADMIN" (sin prefijo ROLE_)
6. **CORS** — el gateway permite localhost:3000, localhost:5173 y localhost:5174
7. **Progreso** — si el usuario no está inscrito, GET progress devuelve 400 (manejar con catch)
8. **imageUrl** — puede ser null si el curso no tiene portada, manejar con fallback

Usuarios de prueba (entorno local y Docker)

Email	Password	Role
admin@edutrack.com	password	ADMIN
david@edutrack.com	123456	USER

Puertos locales (sin Docker)

Servicio	Puerto
API Gateway	8080

Servicio	Puerto
auth-service	8081
course-service	8082
enrollment-service	8083
Frontend (Vite)	5173

Puertos Docker

Servicio	Puerto
API Gateway	8080
Frontend (Nginx)	3000